Введение в веб-технологии часть 3

MySQL и PHP

Вы уже умеете взаимодействовать с базой данных с помощью phpMyAdmin. В этом разделе пойдет речь о том, как взамиодействовать с базой данных с помощью языка программирования PHP.

Для работы с MySQL используется специальное расширение PHP под названием MySQLi (MySQL Improved), реализованное семейством стандартных функций. Через них можно взаимодействовать с базой данных MySQL — выполнять те же самые запросы, которые вы уже выполняли вручную, используя phpMyAdmin.

Хорошая новость ! Расширение `mysqli` доступно «из коробки», иными словами, вам не нужно ничего дополнительно ставить или настраивать.

Чтобы в дальнейшем все было кристально ясно, рассмотрим последовательность работы с MySQL из PHP-сценария (Рис. 48)

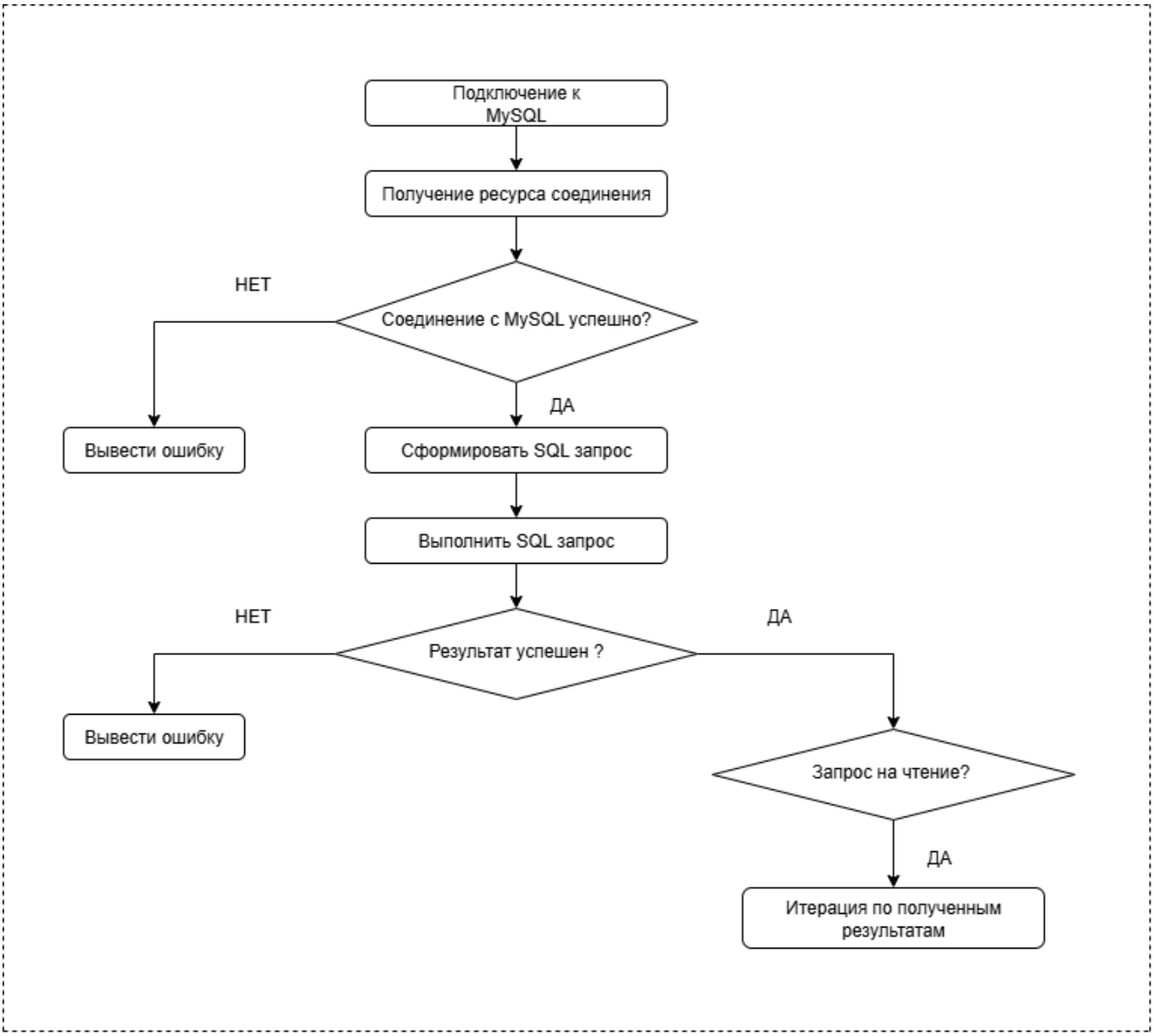


Рис. 48 Последовательность работы с MySQL с помощью PHP

Сначала необходимо установить соединение с сервером базы данных. После этого важно проверить, успешно ли оно прошло. Если подключение установлено, можно переходить к отправке SQL-запросов.

Прежде чем отправить запрос, его нужно составить, затем — передать на выполнение. На завершающем этапе мы получаем и обрабатываем результат выполнения запроса.

Эти шаги являются стандартными и повторяются в любом PHP-скрипте, который работает с базой данных.

Когда вы работали с MySQL через phpMyAdmin, проверка подключения происходила автоматически. В PHP такого интерфейса нет, поэтому проверку подключения необходимо реализовать вручную — с помощью кода и условных операторов.

Далее следует этап формирования запросов. Он считается самым простым, потому что здесь вы используете SQL-запросы, с которыми уже имели дело.

После этого с помощью функций библиотеки `mysqli` запросы отправляются на выполнение. В зависимости от типа запроса, либо работа завершается на этом этапе, либо вы дополнительно обрабатываете полученные данные — например, преобразуете результат в массив записей.

Чтобы стало еще понятнее, посмотрим на тот же процесс под другим углом - глазами пользователя сайта.

Когда пользователь открывает страницу сайта, он на самом деле запускает выполнение PHP-сценария. Этот сценарий первым делом устанавливает соединение с базой данных, вызывая функцию `mysqli_connect()`. Она используется для подключения к серверу базы данных и открытия с ней соединения. В ответ база данных создаёт соединение и передаёт PHP доступ к нему посредством возврата ресурса соединения.

После установления соединения скрипт отправляет SQL-запрос к базе — например, запрос на получение списка стран. Это делается через функцию `mysqli_query()`, которая передаёт в MySQL команду `SELECT`. База данных обрабатывает запрос, находит нужные записи и возвращает результат — специальный объект-ресурс, содержащий найденные данные.

На следующем этапе PHP обрабатывает полученный ресурс, преобразуя его в массив. Как правило, это двумерный массив, где каждая строка соответствует одной записи. После этого массив можно использовать в шаблоне для генерации HTML — в результате пользователь видит список стран на веб-странице.

Всё это происходит мгновенно, буквально за долю секунды, и с точки зрения пользователя выглядит как обычная загрузка страницы с готовым контентом.

Создайте в директории с нашим сайтом файл working_with_db.php и наполните его следующим содержимым:

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8"/>.
    <link href="style.css" rel="stylesheet">
    <title>Московский Государственный Технический Университет им. Н.Э. Баумана</title>
  </head>
  <body>
    <header>
        <div class="logo">
          <img class="logo__image" src="img/logo_with_text.png" alt="Логотип МГТУ" width="275" height="70">
        </div>
        <div class="navigation">
          <ul class="navigation__list">
            <li class="navigation__list-item">
```

```
              <a class="navigation__link" href="index.html">Главная
страница</a>
            </li>
            <li class="navigation__list-item">
              <a class="navigation__link" href="#">Контакты</a>
            </li>
          </ul>
        </div>
    </header>
  <main>

    <h1>Работа с базой данных</h1>

    // писать код дальше будем здесь
    <?php ?>

  </main>
  <footer>
    <?php МГТУ им. Н.Э. Баумана © Все права защищены ?>
  </footer>
  </body>
</html>
```

Весь последующий код на PHP будем писать после тэга `<h1>` внутри `<main>`.

На всякий случай напоминаем, что у вас должен быть установлен XAMPP, а файл с расширением .php должен создаваться внутри папки с сайтом в директории htdocs.

Инструкция по установке XAMPP находится в разделе "Настройка рабочего окружения" внутри главы про язык PHP.

Все готово, давайте же начнем программировать!

Подключение к MySQL

Для работы с БД к ней сначала необходимо подключиться. В языке PHP это делается с помощью функции `mysqli_connect`, принимающая следующие аргументы: <хост>, <пользователь>, <пароль>, <имя бд>.
Если пароль не задан, то на его месте надо передать пустые кавычки.

```
<?php
$con = mysqli_connect("localhost", "root", "","world_data");
```

Функция подключения к базе данных возвращает либо ресурс соединения при успешном подключении, либо, в случае неудачи вернет значение false.
В коде важно предусмотреть обработку обеих описанных ситуаций.

При неудачном подключении к БД покажем текст ошибки пользователю:

```
<?php
$con = mysqli_connect("localhost", "root", "", "world_data");

if ($con == false) {
   print("Ошибка подключения: " . mysqli_connect_error());
}
else {
   print("Соединение установлено");
   print("<br/>");
    // выполнение запросов
}
```

Здесь инструкция `print("<br/>");` добавляет на страницу тэг `<br/>` , который используется для создания переноса строки.

Обратите внимание, мы вставляем html тэги с помощью PHP.

На рисунке 49 изображена страница с уведомлением об успешном подключении к БД.

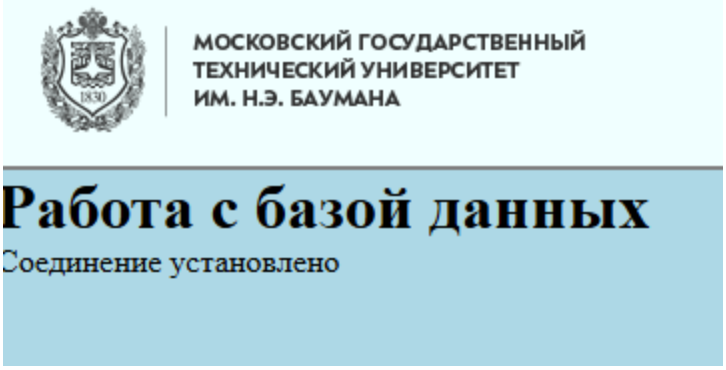


Рис. 49 Фрагмент страницы с уведомлением об успешном подключении к БД.

Если подключение вернуло false, то мы пытаемся понять что конкретно пошло не так при попытке подключения. Для этого мы используем функцию `mysqli_connect_error()`, которая возвращает строку, в которой написан текст ошибки. Попробуйте намеренно изменить один из параметров функции `mysqli_connect()` и посмотреть что будет выведено на страницу.

Выполнение запросов

Мы уже умеем работать с языком SQL в phpMyAdmin. Говоря конкретнее, мы умеем делать запросы. Мы делали разные запросы, одни помогали нам получить какие-то данные из БД, другие создавали новые записи, а третьи что-то удаляли.

Все SQL-запросы можно разбить на 2 группы: на чтение и на модификацию.

Запросы на чтение

Эти запросы очень просто обнаружить - они начинаются со слова `SELECT`. Если мы

используем такие запросы, то ожидаем получить от них список записей. Исполнение таких запросов через `mysqli` будет возвращать ресурс результата.

Запросы на модификацию
К ним относятся все остальные SQL-запросы: `UPDATE`, `INSERT`, `DELETE`. При исполнении этих запросов всегда ожидается сообщение о том, что всё прошло успешно или произошла ошибка. Запросы на модификацию, будут возвращать либо `true` либо `false`.

Давайте выполним какой-нибудь простой запрос к БД используя язык PHP, например, добавим новую страну в нашу таблицу.

Составим запрос. Он может выглядеть так:

```
INSERT INTO countries SET capital = 'Canberra', country = 'Australia'
```

или так:

```
INSERT INTO countries (country, capital)
VALUES ('Australia', 'Canberra');
```

Выполнять SQL-запросы на PHP очень просто - для этого используется функция `mysqli_query()`.

Эта функция принимает 2 аргумента - первым аргументом является ресурс соединения, полученный ранее из функции `mysqli_connect()`, а вторым аргументом передается текст SQL-запроса в сиду строки.

Запрос `INSERT` является запросом на модификацию, поэтому мы ожидаем, что функция вернет `true` или `false`.

Изменим наш код на странице:

```
<?php
    $con = mysqli_connect("localhost", "root", "", "world_data");
    if ($con == false) {
        print("Ошибка подключения: " . mysqli_connect_error());
    }
    else {
        print("Соединение установлено");
        print("<br/>");
        $sql = "INSERT INTO countries SET capital = 'Canberra', country =
'Australia'";
        $result = mysqli_query($con, $sql);
    }
?>
```

Продолжаем обрабатывать ошибки

Проверять нужно не только коррекность подключения, но также и правильность исполнения запроса,, ведь он тоже может завершиться с ошибкой.

Ошибки могут быть самые разные. Например, вы ошиблись в синтаксисе SQL-запроса или сервер может стать недоступен уже после подключения.

Нужно всегда проверять выполнение запроса. Хотя бы для того, чтобы на сайте показать информацию вида «ваша запись добавлена» и пользователь знал, что его данные сохранились.

Как проверить ошибку?

Если после исполнения запроса функция вернула `false`, то это означает, что запрос не исполнился, а значит произошла ошибка.

Нужно поставить условие на результат выполнения этой функции, обработать и показать сообщение об ошибке на сайте.

Допишем наш код:

```
<?php

    $con = mysqli_connect("localhost", "root", "", "world_data");

    if ($con == false) {

        print("Ошибка подключения: " . mysqli_connect_error());

        print("<br/>");

    }

    else {

        print("Соединение установлено");

        print("<br/>");

        $sql = "INSERT INTO countries SET capital = 'Canberra', country =
'Australia'";

        $result = mysqli_query($con, $sql);

        if (!$result){

            $error = mysqli_error($con);
```

```
            print("Ошибка MySQL: " . $error);

            print("<br/>");
        }
        else {

            print("Запись добавлена");

            print("<br/>");
        }

    }
?>
```

Запросы на чтение данных

Мы научились делать на добавление записей. Но гораздо чаще на сайтах используются запросы на чтение.

Точно также, как запросы на модификацию, запрос на чтение нужно сначала сформировать, а потом выполнить.

Важно обратить внимание на то, что запросы на чтение данных возвращают не `true` или `false`, а **ресурс результата** либо `false`, если произошла ошибка.

Ресурс результата при необходимости преобразовывают в массив, чтобы отобразить на странице.

Остановимся на понятии ресурса (или объекта) результата подробнее.

Ключевой момент:
функция `mysqli_query()` при выполнении запроса на получение данныз возвращает не массив данных, а специальный ресурс (объект) результата. Этот ресурс представляет собой некое представление результата запроса в памяти базы данных.

Чтобы получить данные в виде привычного массива, двумерного или ассоциативного, этот результат нужно вызовом другой функции преобразовать в массив.

Пример кода, реализующего получение записей:

```
<?php

    $con = mysqli_connect("localhost", "root", "", "world_data");

    if ($con == false) {

        print("Ошибка подключения: " . mysqli_connect_error());
```

```
        print("<br/>");

    }
    else {

        print("Соединение установлено");

        print("<br/>");

        $sql = "SELECT id, country, capital FROM countries;";

        $result = mysqli_query($con, $sql);

        if (!$result){

            $error = mysqli_error($con);

            print("Ошибка MySQL: " . $error);

            print("<br/>");
        }
        else {

            $rows = mysqli_fetch_all($result, MYSQLI_ASSOC);

            foreach ($rows as $row) {

                print("Страна: " . $row['country']. " столица: "  .
$row['capital']);

                print("<br/>");
            }
        }

    }
?>
```

Необольшие пояснения к коду:
Сначала в коде выполняется подключение к базе данных вместе с проверкой его корректности.

Далее у нас имеется запрос типа `SELECT`, который мы выполняем используя функцию `mysqli_query()`.

После этого мы проверяем корректность выполнения запроса. Если запрос выполнился без ошибок, то в переменной `$result` находится ресурс результата.

Чтобы получить список стран и их столиц в виде массива, нужно полученный ресурс преобразовать вызовом функции `mysqli_fetch_all()`. Эта функция берет ресурс и возвращает классический двумерный массив, который мы обходим с помощью `foreach` и печатаем на экран названия стран и их столиц.

Поговорим немного подробнее про функцию `mysqli_fetch_all()`.
Функция принимает 2 аргумента, первый из которых обязательный и представляет собой ресурс результата, второй, необязательный, определяет формат возвращаемых данных:

- `MYSQLI_ASSOC` - возвращает ассоциативный массив (ключи - имена столбцов)
- `MYSQLI_NUM` - возвращает индексированный массив (ключи - числовые индексы)
- `MYSQLI_BOTH` - возвращает массив с обоими типами ключей

Получается, что с запросами типа `SELECT` в нашем коде добавляется ещё одно действие — преобразование результата в массив.

SQL инъекция

В начале нашего курса мы говорили о GET- и POST-запросах. В частности, мы обращали внимание на то, что информация в GET-запросе передаётся прямо в заголовке.

```
GET /blog/?name1=value1&name2=value2
HTTP/1.1
```

То, что информация передаётся прямо в заголовке означает, что параметры запроса являются частью адреса страницы.

Приведем еще один пример, чтобы стало немного понятнее. Предположим, что вы хотите найти в гугл информацию об МГУ.

Для этого введите в адресную строку браузера вот эту строку:

```
http://www.google.com/search?q=MSU
```

Теперь вы хотите найти информацию об МГТУ им. Н.Э. Баумана.

Для этого введите в адресную строку браузера вот эту строку:

```
http://www.google.com/search?q=BMSTU
```

Видите, мы меняем параметр `q`.

Напомним еще один важный факт, о котором мы говорили когда изучали обработку форм на PHP — полученные от пользователя данные автоматически сохраняются в двух глобальных ассоциативных массивах:

- `$_POST`, если форма отправлена методом `POST`;
- `$_GET` — используется для получения данных, переданных методом `GET`;

Напишем очень простой код, который проиллюстрирует это. Создадим файл под названием working_with_get_request.php и напишем в него код, обрабатывающий GET-запрос.

```
<?php

if (isset($_GET['country'])) {

    print("Название страны, переданное в запросе: " . $_GET['country']);

    print("<br>");

}
else {

    print("Данные в параметре country через GET-запрос не переданы");

}
?>
```

При выполнении такого запроса:

```
http://localhost/BMSTU_website/working_with_get_request.php?
country=test1234567
```

На странице отобразится следующее (Рис. 50):

Название страны, переданное в запросе: test1234567

Рис. 50 Результат запроса

Когда вы отправляте форму с сайта, вы делаете GET или POST запрос, в зависимости от значения атрибута `action` тэга `<form>`

Таким образом можно передать внутрь сценария значение, которое ввёл пользователь. Затем в самом сценарии можно получить это значение адресной строки, обратившись к массиву `$_GET` или `$_POST` соответственно.

Часто значения от пользователей в параметре запроса используются для формирования SQL-запроса.

Например, мы хотим реализовать простой, но полезный сервис, который будет работать следующим образом: пользователь вводит название страны и получает в ответ столицу страны, которую он ввел.

Для создания такого сервиса наш скрипт должен получать от пользователя, неважно, через форму или через строку браузера название страны в виде параметра GET- или POST-запроса, подставлять его в SQL-запрос и возвращать пользователю столицу этого государства.

Без подстановки пользовательского значения страны в запрос мы никак не сможем показать ему столицу потому что для ее поиска нам нужно сформировать запрос `SELECT` и в `WHERE` передать то, что ввел пользователь.

Для того, чтобы получить столицу Российской Федерации нам нужно выполнить такой запрос:

```
SELECT capital FROM countries WHERE country = 'Russia';
```

Для того, чтобы получить столицу страны, которую ввел пользователь нам написать вот так:

```
SELECT capital FROM countries WHERE country = $_GET['country'];
```

или вот так:

```
SELECT capital FROM countries WHERE country = $_POST['country'];
```

Еще раз, пользователь инициирует запрос на сервер используя форму обратной связи, скрипт получает доступ к нужному параметру запроса, параметр подставляется в SQL-запрос и выполняется скриптом.

Именно здесь нам следует задуматься о безопасности.

Если мы об этом не подумаем, то об этом точно подумает злоумышленник и сможет выполнить атаку типа SQL-инъекция.

Возможность эксплуатации этой уязвимости возникает при совместной работе PHP и MySQL, а именно при смешивании данных и кода.
Когда мы подставляем значение в запрос, то мы смешиваем данные и код запроса.

> Важно понимать, что эта уязвимость не характерна именно для PHP и MySQL. Она может возникнуть во всех случаях, когда код на сервере небезопасно обрабатывает пользовательский ввод при формировании запросов в базу данных. Иными словами, во всех случаях, когда данные и код смешиваются в подобном контексте.

Код логики запроса - это все, что не относится к данным: например ключевое слово `SELECT`, имя таблицы, имена полей.

Значение - это изменяемая часть, которая присваивается запросу динамически в момент его исполнения. В этот момент возникает возможность для атаки вида SQL-инъекция.

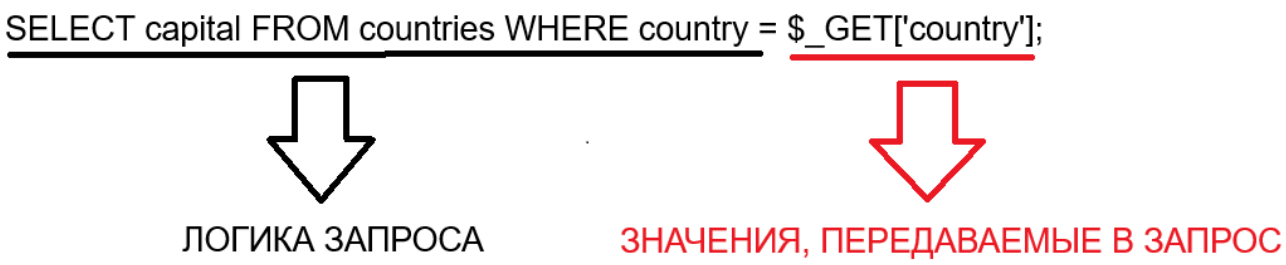


Рис. 51 Схематичное изображение SQL-запроса

Когда веб-приложение напрямую вставляет пользовательские данные в SQL-запросы без должной проверки и обработки, это создает для злоумышленника множество возможностей. Хакеры могут манипулировать такими запросами, полностью изменяя их исходный смысл и функциональность.

Используя эту уязвимость хакер может полностью уничтожить базу данных (например выполнив запросы типа `DROP TABLE`) , он может украсть приватные данные пользователей, может назначить себя администратором, он может подменить содержимое базы, включая изменение прав доступа и паролей.

Как вы видите, это очень опасная уязвимость. Давайте проэксплуатируем ее на примере.

Создайте файл sqli_example.php и наполните его этим кодом:

```php
<?php
$con = mysqli_connect("localhost", "root", "", "world_data");

$id = $_GET['ID'];

$sql = "DELETE FROM countries WHERE ID = '$id'";

$result = mysqli_query($con, $sql);
?>
```

Это пример плохого PHP-кода, который допускает добавление инъекции в SQL-запрос. В данном коде параметр, получаемый от пользователя записывается в переменную `$id`. Затем это значение подставляется в запрос.
На первый взгляд код работает нормально и в нем нет ничего опасного.

Запрос

```
http://localhost/BMSTU_website/sqli_example.php?ID=12
```

удалит из таблицы countries запись с id=12.

Хакер подставил в параметр запроса вместо числа строку, которая поменяла смысл SQL-выражения.

Вместо удаления одной записи с переданным ID, модифицированный запрос удалил все записи в таблице.

```
http://localhost/BMSTU_website/sqli_example.php?ID=11' OR 1=1 -- '
```

Прокомментируем запрос:

- `11'` - закрывает строковый параметр в исходном запросе
- `OR 1=1` - добавляет условие, которое ВСЕГДА истинно
- `-- '` - комментирует остаток оригинального запроса (включая закрывающую кавычку)

Финальный запрос будет интерпретирован так:

```
DELETE FROM countries WHERE ID = 11 OR 1=1
```

Поскольку условие `1=1` всегда истинно, запрос игнорирует первоначальное условие `id = 11`, удаляет ВСЕ записи из таблицы countries

Способы защиты от SQL-инъекции

Чтобы защитить свой SQL-запрос от хакеров необходимо принудительно приводить значения к нужным типам (в данном случае к числовому типу), а также выполнить экранирование опасных символов.

Самый простой вариант — принудительно привести значение к определённому типу. Как в случае с удалением страны по её ID, мы на месте ID всегда ожидаем получить число. Если ожидается только число, то возможно принудительно, что бы мы ни получили, привести это к числу. И тогда, если хакер передаст строку, то эта строка будет преобразована к нулю, а нуль ничего плохого не сделает.

Вариант номер два более универсальный. Когда мы работаем со строкой, то можем опасные символы, к ним в первую очередь относятся ограничительные символы (кавычки), экранировать. Так, чтобы они перестали быть ограничителями, а стали считаться значениями.

Приведение к числовому типу

В нашем SQL-запросе ожидается использование числовых значений. В этом случае имеет смысл принудительно привести значение к числовому типу.

`DELETE FROM countries WHERE ID = <ИДЕНТИФИКАТОР>`

Когда мы работаем с условием по первичному ключу, то если первичный ключ — это число, соответственно это значение можно заранее привести к числу. Проделав это, мы избежим в этом значении любой SQL-инъекции.

```
<?php
$id = intval($_GET['ID']);

$sql = 'DELETE FROM countries WHERE ID = ' . $id;
```

Измените код и проверьте, что защита работает.

Экранирование данных

Приведение к типу не сработает, когда обрабатываемое значение не числовое, а строковое.

В этом случае требуется комплексный подход. Зная механизм работы SQL-инъекции, мы можем преобразовать все символы, которые позволяют её сделать, к их безопасным аналогам.

Кавычки заменяются на их аналог, который больше не трактуется как ограничитель. Например, если в значении есть кавычка, то после экранирования эта кавычка заменится на обратный слеш с кавычкой. Эта пара символов сохранится в базе данных, и будет показываться в тексте как нормальная кавычка. Но перестанет трактоваться как управляющий символ в SQL-запросе, то есть как ограничитель.

Без экранирования

`INSERT INTO countries (country, capital) VALUES ('Côte d'Ivoire', 'Yamoussoukro')`

С экранированием

`INSERT INTO countries (country, capital) VALUES ('Côte d\'Ivoire', 'Yamoussoukro')`

Для работы с экранированием есть специальная функция — `mysqli_real_escape_string()`.

Вы передаёте ей ресурс соединения и то значение, которое хотите экранировать, то есть то значение, которое будет подставлено в запрос.

Это значение в итоге преобразуется к безопасному виду, и уже его можно подставлять в сам запрос и исполнять. То есть мы фильтруем значения перед их добавлением в SQL.

```php
<?php

$con = mysqli_connect("localhost", "root", "", "world_data");

$safe_country = mysqli_real_escape_string($con, $_GET['country']);

$sql = "DELETE FROM countries WHERE country = '$safe_country'";

$result = mysqli_query($con, $sql);
```

Подготовленные выражения

Проблема традиционного подхода к формированию SQL-запросов в том, что данные и код запроса смешивались, и поэтому становится возможной эксплуатация уязвимости типа SQL-инъекция.
Если же значения отправлять отдельно от кода, изолировать эти две сущности, то для таких манипуляций не останется места.

Как работают подготовленные выражения

Работу подготовленных выражений условно можно разделить на 4 этапа:

1. Сначала вы отправляете "каркас" запроса, где вместо реальных значений стоят специальные метки-заполнители. В роли этих меток выступают символы `?`.
   Это похоже на отправку анкеты с пустыми полями для ответов.
2. Сервер базы данных получает этот шаблон и анализирует его структуру, но не выполняет. На этом этапе определяются таблицы и условия выборки, создается "форма" для последующего заполнения
3. Затем отдельно передаются фактические данные, которые заполняют подготовленные "ячейки" в шаблоне, всегда обрабатываются как значения, а не как часть SQL-кода и дополнитеьно проходят строгую проверку типов.
4. Только после этого полностью сформированный запрос выполняется, гарантируя:
   - Безопасность (данные не могут стать кодом)
   - Эффективность (шаблон компилируется один раз)
   - Корректность (выполняется строгая проверка типов данных)

Такой подход исключает любую возможность SQL-инъекций, так как данные и код запроса физически разделены на всех этапах обработки.

Научимся работать с подготовленными выражениями на простом примере:

```php
<?php

$con = mysqli_connect("localhost", "root", "", "world_data");

$country = $_GET['country'];
```

```php
$sql = "DELETE FROM countries WHERE country = ?";

$stmt = mysqli_prepare($con, $sql);

mysqli_stmt_bind_param($stmt, 's', $country);

mysqli_stmt_execute($stmt);

?>
```

В `mysqli` есть специальный набор функций для использования подготовленных выражений.

Пусть у нас есть значение, полученное из формы. Формируем строку SQL-запроса с плейсхолдером вместо значения.
Передаём её в функцию `mysqli_prepare()`. Эта функция вернёт объект подготовленного выражения.
В этот объект мы подставляем значения с помощью функции `mysqli_stmt_bind_param()`. И далее мы выполняем полностью сформированный запрос `mysqli_stmt_execute()`.

При использовании подготовленных выражений обязательно указывать тип данных для каждого параметра. Нужно определить, какой тип у каждого передаваемого значения.

Типы значений:

Символ	Описание
i	соответствующая переменная имеет тип integer
d	соответствующая переменная имеет тип double
s	соответствующая переменная имеет тип string
b	соответствующая переменная является большим двоичным объектом (blob) и будет пересылаться пакетами

Если передаётся сначала число, а потом строка — в параметре, отвечающем за типы, нужно передать `is` соответственно, а если передается 2 строки, то `ss`.

Чтобы получить объект результата после выполнения подготовленного выражения например для запроса на получение данных, используйте функцию `mysqli_stmt_get_result`. Она вызывается после `mysqli_stmt_execute()`, принимает подготовленное выражение и возвращает ресурс результата, то, что раньше возвращала функция `mysqli_query()`.

Перепишите и выполните пример:

```php
<?php

$sql = "SELECT id, country, capital FROM countries WHERE id = ?";

$stmt = mysqli_prepare($con, $sql);

mysqli_stmt_bind_param($stmt, 'i', $_GET['id']);

mysqli_stmt_execute($stmt);

$res = mysqli_stmt_get_result($stmt);

$rows = mysqli_fetch_all($res, MYSQLI_ASSOC);

foreach ($rows as $row) {

    print("Страна: " . $row['country']. " столица: "  . $row['capital']);

    print("<br/>");

}

?>
```